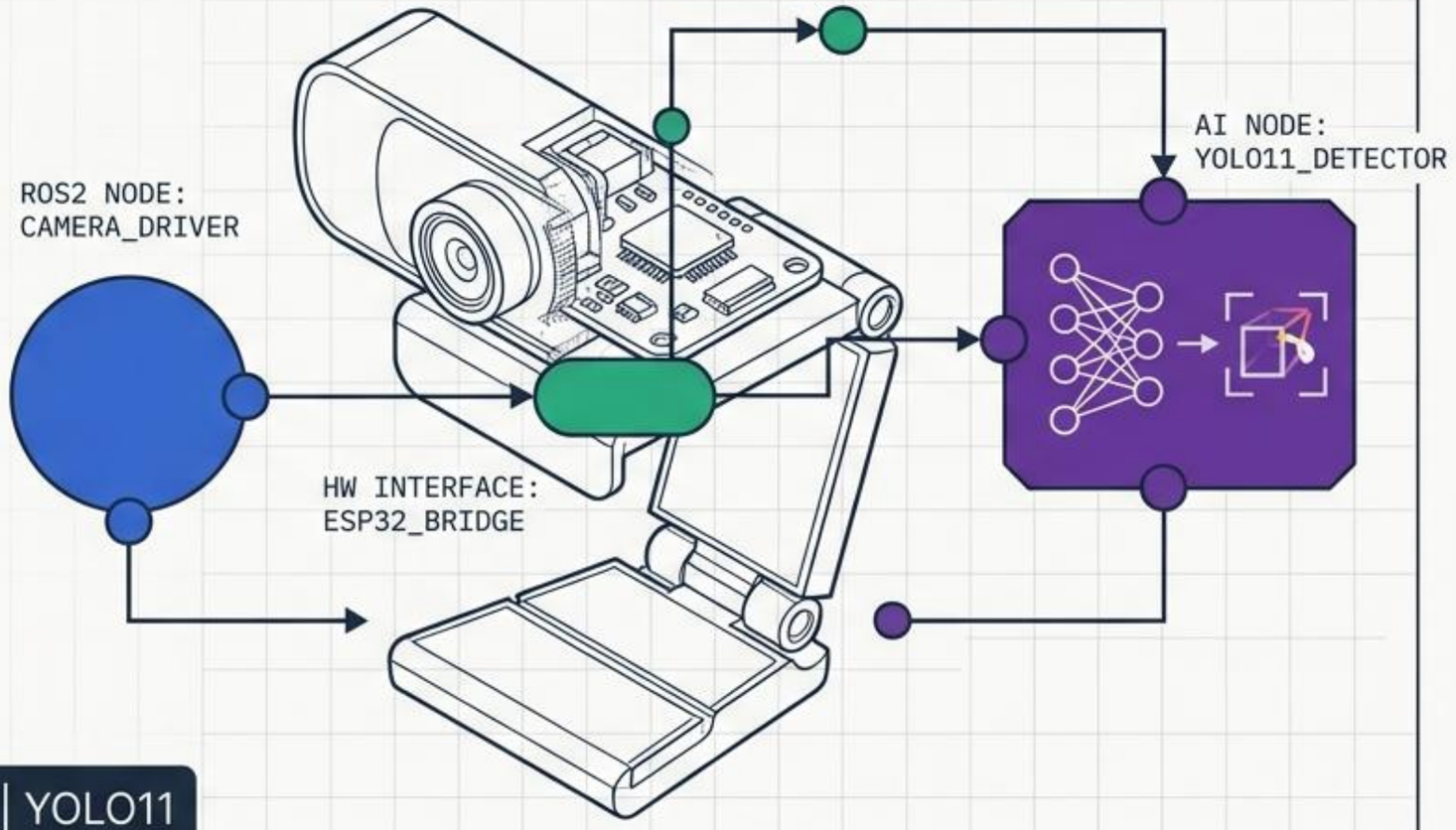
TRIC MindSpark Series: ROS2 Hands-On Integration

From First Node to Custom **YOLO11 Deployment**



Ubuntu 24.04 | ROS2 Jazzy | OpenCV | ESP32 | YOLO11

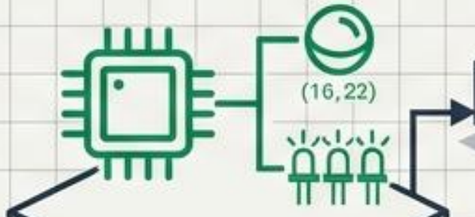
Gemini Notebook

One Evolving Package, Five Increasingly Capable Systems

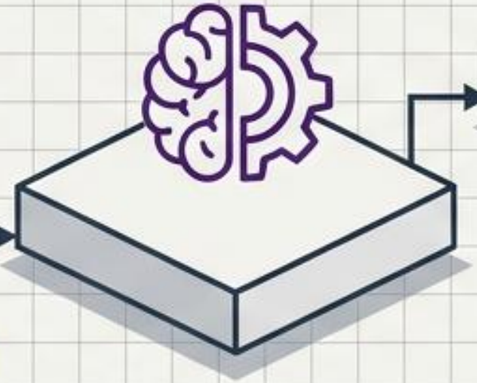
PW1: `hello_node` (Build a workspace, Python package, and first node)

PW2: Shared camera & independent color detectors

PW3: Hardware translation (Green-ball location to ESP32 LEDs)

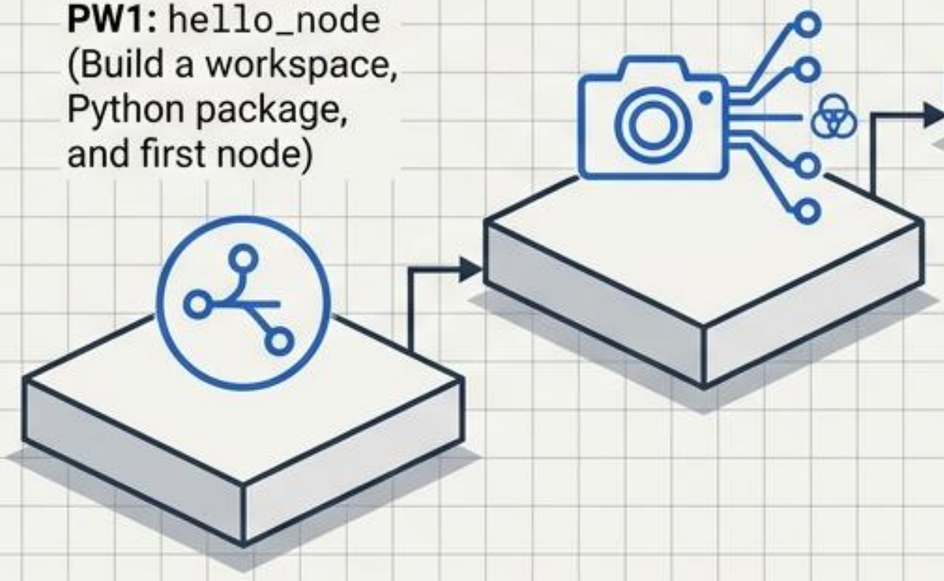


PW4: Pretrained YOLO11 (Reuse nodes via remapping)



PW5: Custom YOLO (Record, annotate, train, deploy)





The Core Constraint:

One Workspace (`~/mindspark_ws`) | **One Package** (`mindspark_robot`).

Each practical extends the previous system instead of replacing it.

Gemini Notebook

The Technical Baseline and Terminal Ritual

Operating System:
Ubuntu 24.04 LTS,
ROS2 Jazzy,
Python 3

Development:
VS Code, colcon,
rclpy, ROS2 CLI

Vision:
USB webcam,
OpenCV, cv_bridge
(320 × 240 baseline)

Hardware:
ESP32 DevKit,
USB serial, Red/
Yellow/Green LEDs

AI Stack:
Ultralytics YOLO11,
PyTorch, local
NVIDIA GPU

Workspace:
~/mindspark_ws
(ROS_DOMAIN_ID=30)

The Terminal Ritual

```
source /opt/ros/jazzy/setup.bash

export ROS_DOMAIN_ID=30
export ROS_AUTOMATIC_DISCOVERY_RANGE=SUBNET

source ~/mindspark_ws/install/setup.bash
```

Gemini Notebook

Building the Foundational ROS2 Node

1. Registration



`setup.py`

Register hello_node in console_scripts. This maps the terminal command to the Python main() function.

2. Compilation



`colcon build --symlink-install`

Creates the installed executable in the workspace tree.

3. Execution



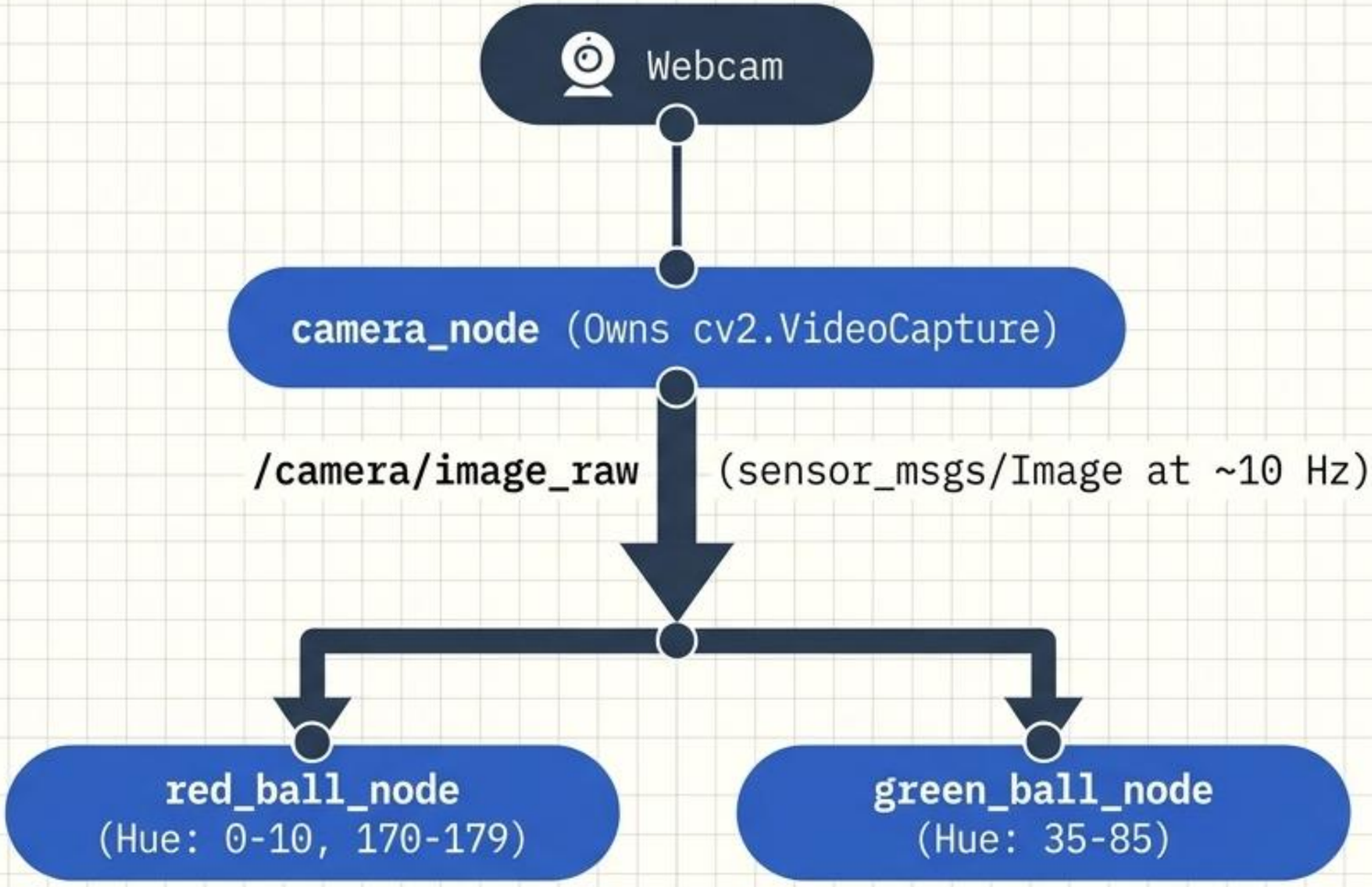
`ros2 run mindspark_robot hello_node`

Starts the independent process participating in the ROS graph.

```
[INFO] [hello_node]: MindSpark ROS2 Robot System Running!
[INFO] [hello_node]: MindSpark ROS2 Robot System Running!
```

Gemini Notebook

Sharing Vision Through Publisher–Subscriber Architecture

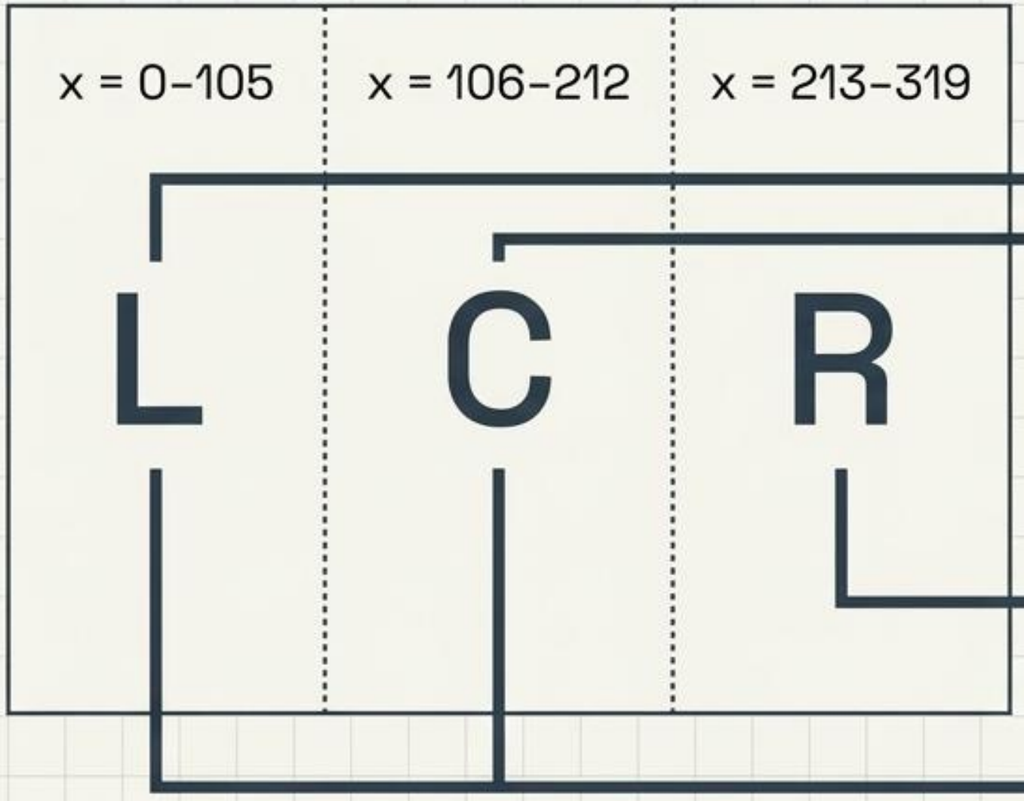


Independent nodes ≠ **duplicated webcams**. One sender supports several parallel subscribers. Topic subscribers must not call `cv2.VideoCapture()` again.

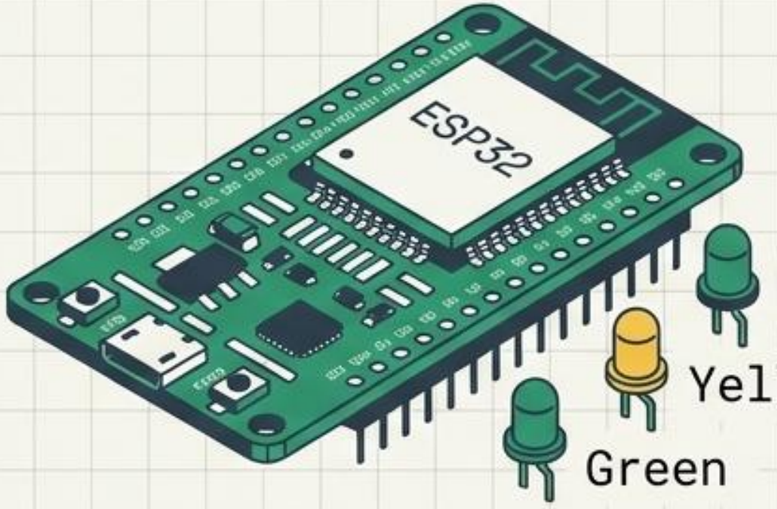
Gemini Notebook

Translating Pixel Geometry to Hardware Actuation

The HUD



The Hardware



L → **GPIO25** (Red LED)

C → **GPIO26** (Yellow LED)

R → **GPIO27** (Green LED)

Failsafe: No detection or a 2-second ROS2 timeout sends command ‘N’ → All LEDs OFF.

Gemini Notebook

The End-to-End Perception and Control Graph

1. Perception (Camera)

`camera_node` publishes `/camera/image_raw`

2. Detection (HSV)

`green_ball_node` publishes text coordinates (e.g., DETECTED: x=160, y=104) to `/green_ball`

3. Decision (Logic)

`vision_decision_node` evaluates the x-coordinate. Publishes L, C, R, or N to `/vision/decision` (Target: 5 Hz).

4. Actuation (Hardware)

`esp32_control_node` subscribes and sends serial commands via `/dev/ttyUSB0` (115200 baud) to the ESP32 LEDs.

Separating perception, decision, and actuation into distinct modules allows for seamless future component swapping.

Gemini Notebook

Fusing Pretrained Object Detection with Color Masks

YOLO: What Object?



Pretrained YOLO11n (COCO Class 32) identifies a sports ball. It provides bounding boxes and confidence scores, but cannot verify color.

+

HSV: Which Color?

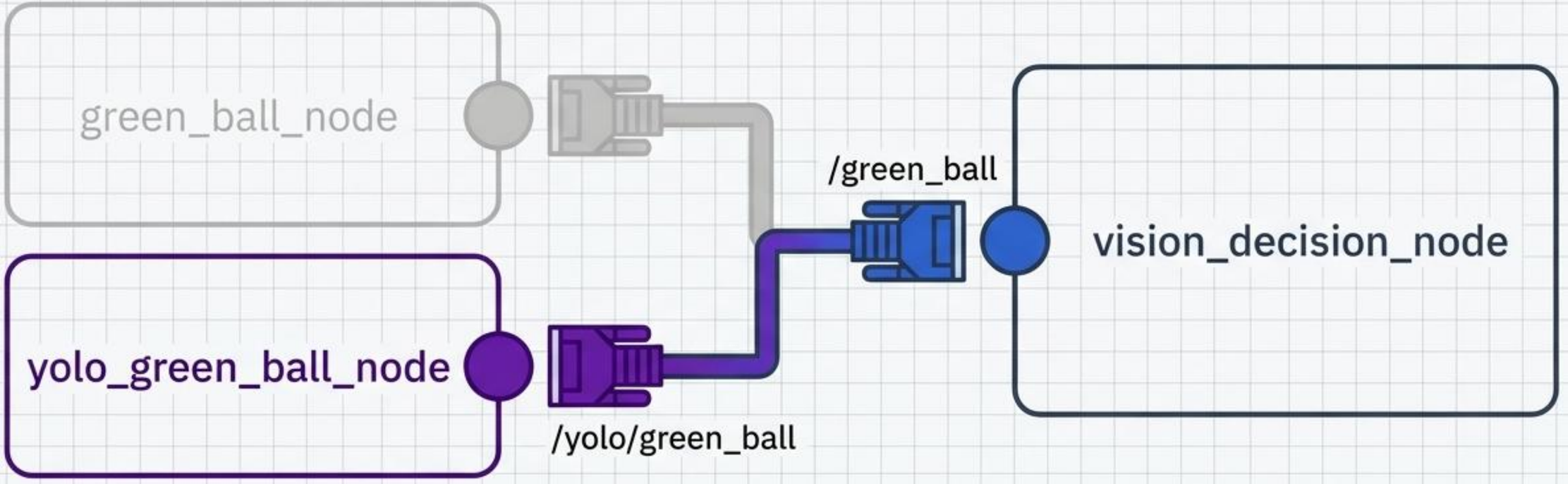


HSV ROI Check.
Verify the color within the YOLO bounding box.

Class 32 (Sports Ball) + Green Pixel Ratio ≥ 0.30 = Accepted Target

Gemini Notebook

Topic Remapping Enables Hardware Reuse Without Rewriting




```
ros2 run mindspark_robot vision_decision_node --ros-args -r /green_ball:=/yolo/green_ball
```

Zero source-code changes to the decision node. The old input name /green_ball now dynamically resolves to the new AI output at runtime.

Gemini Notebook

The Custom AI Training and Deployment Pipeline



1. Record (ROS2)

5 MP4 clips from /camera/image_raw (~7.68 FPS)

2. Extract

Offline OpenCV processing (353 JPEGs)

3. Annotate

Roboflow single-class (green_ball), 351 tight bounding box labels

4. Split

Segregate by video recording

5. Train

YOLO11n locally on RTX 2060 GPU (50 epochs, 640x640)

6. Evaluate

Held-out test split

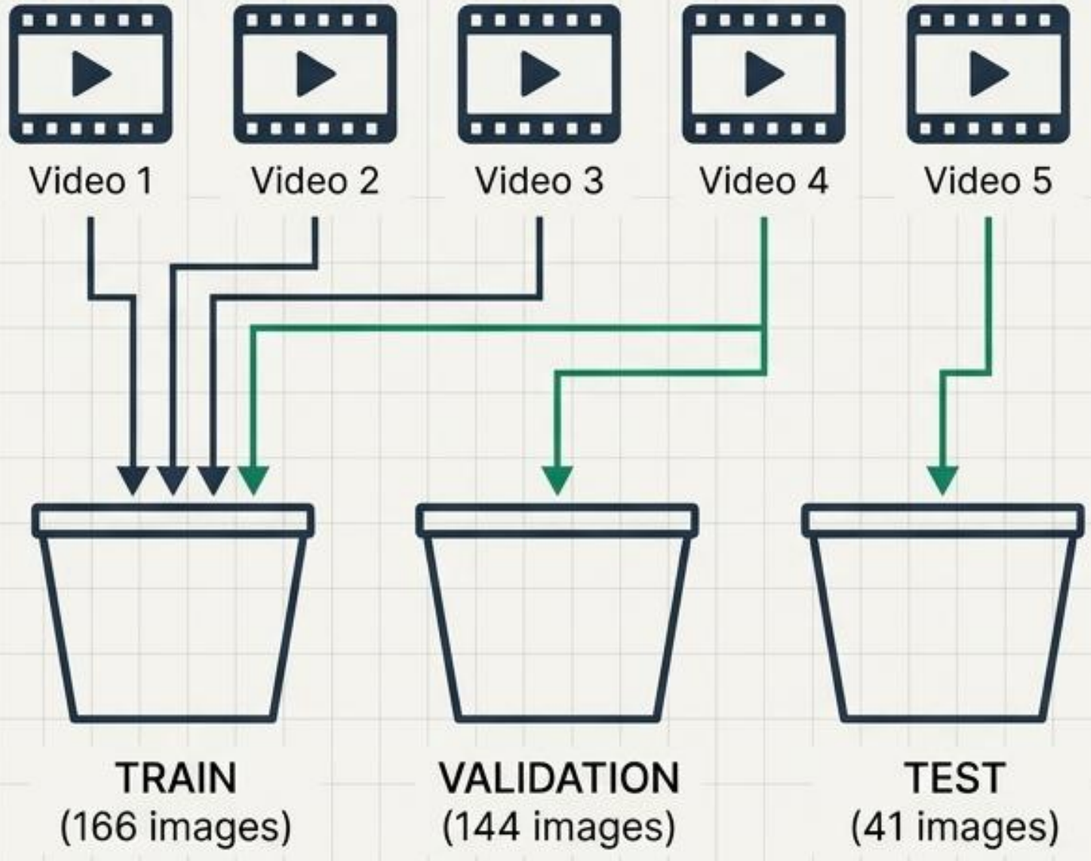
7. Deploy

Custom ROS2 executable

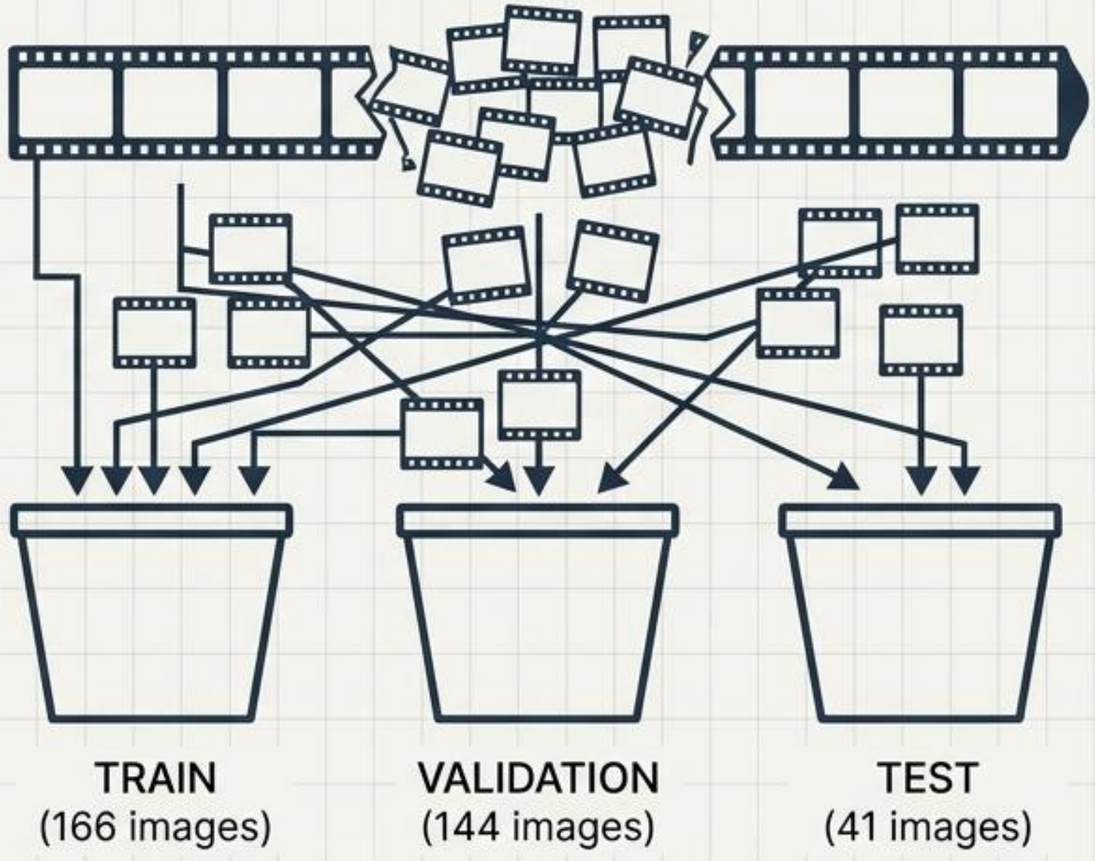
Gemini Notebook

Session-Aware Data Splitting Prevents Model Leakage

✓ The Correct Way



✗ The Wrong Way



Correlated frames from a single continuous recording must not cross partitions.
Recording independence takes priority over achieving exact conventional split ratios.

Gemini Notebook

Quantitative Metrics Versus Real-World Edge Cases

Quantitative Test (Lab)

Precision:	0.998
Recall:	1.000
mAP50:	0.995
mAP50-95:	0.823

Context: Tested on 41 images from a distinct, separate recording session in a controlled environment.

Live Challenge (Reality)

- **Observation:** Non-green balls were occasionally predicted as `green_ball` targets.
- **Context:** High test mAP does not guarantee real-world color robustness under varying lighting or against novel distractors.

Validation metrics guide model development; held-out tests prove baseline capability; live physical testing reveals edge cases requiring hard-negative data.

Gemini Notebook

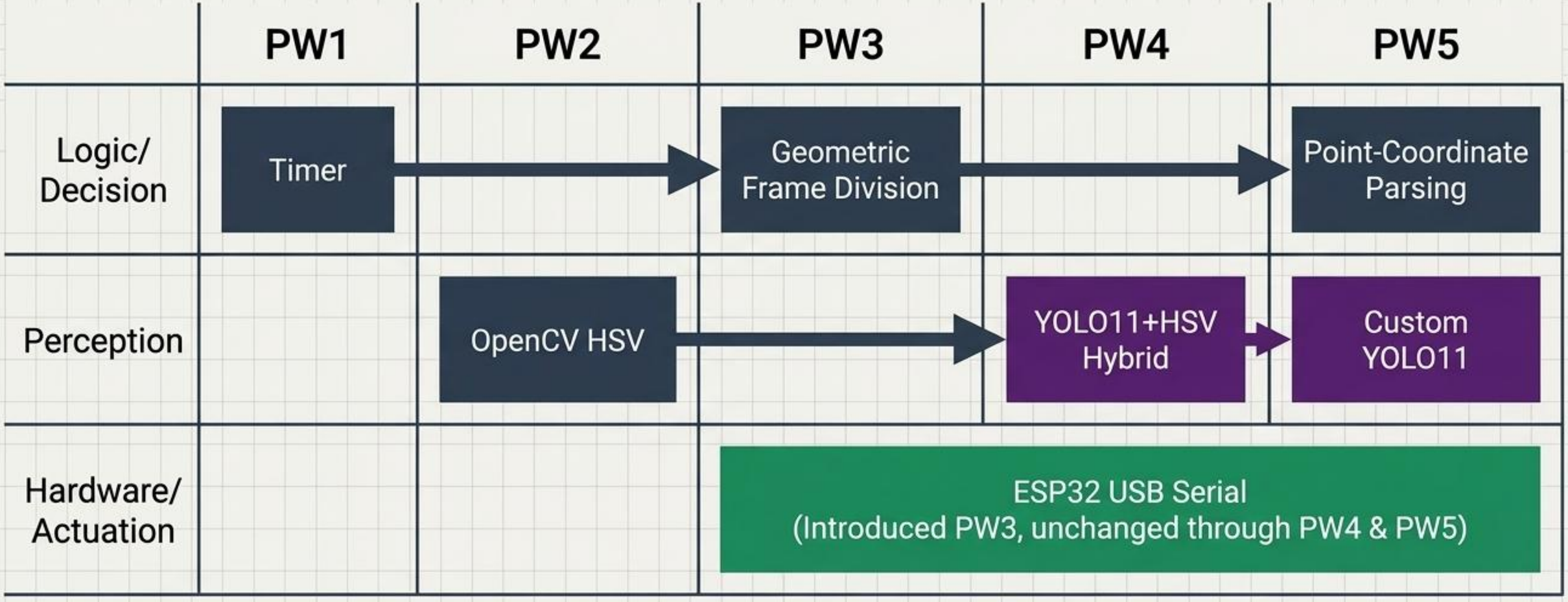
Deploying the Custom Inference Interface



Using **geometry_msgs/Point**.
Point.x and **Point.y** provide the pixel center.
Point.z is hijacked to transmit confidence score (NOT real-world depth).

Gemini Notebook

Retrospective: The Power of Modular System Architecture



By strictly separating perception, decision-making, and actuation, we created an evolving robotic system where AI could be seamlessly injected without rewriting hardware control loops.

Gemini Notebook

Core Engineering Capabilities Acquired

ROS2 Development

Workspaces, ament_python packages, node lifecycles, and console_scripts.

Architecture & Comms

Publishers, subscribers, domain discovery, and topic remapping CLI.

Computer Vision

Shared camera management, cv_bridge, and HSV pixel geometry.

Hardware Integration

Serial messaging, ESP32 GPIO logic, and modular decision layers.

Deep Learning

YOLO11 inference, dataset curation, and venv interpreter management.

Engineering Practice

Combating data leakage, end-to-end throughput verification, and live testing vs. lab metrics.

Gemini Notebook